Tim Pengajar IF2150

IF2150 – Rekayasa Perangkat Lunak

Software Product Line

SEMESTER I TAHUN AJARAN 2025/2026

KNOWLEDGE & SOFTWARE ENGINEERING

Product Line Engineering

- Idea:
  - Production Line in Ford Model T

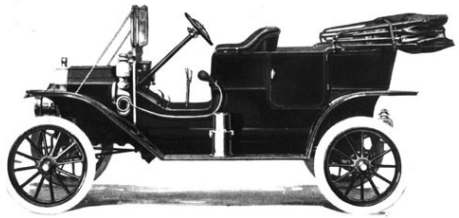
- **Individual Production vs Mass Production**
- Which one costs more?
- Which one takes less time to develop?
- Which one is easy to customize?

KNOWLEDGE & SOFTWARE ENGINEERING

Ford Model T Production Line

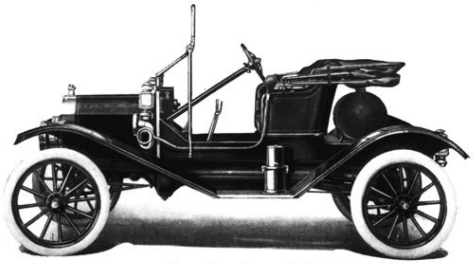
KNOWLEDGE & SOFTWARE ENGINEERING

A Complete Line of Model T's to Choose From



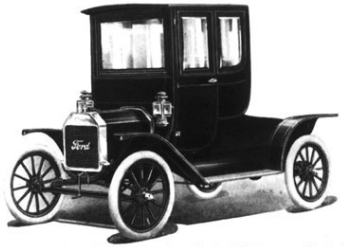
5-Passenger Touring Car, Fully Equipped

3-Passenger Roadster, Fully Equipped

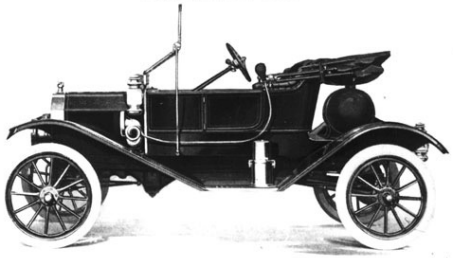


2-Passenger Open Runabout, Fully Equipped

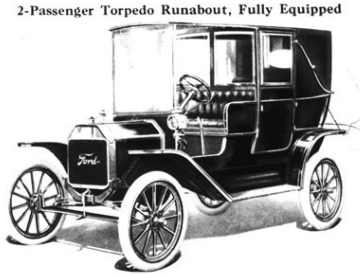
Ford Car Models Supply Every Demand



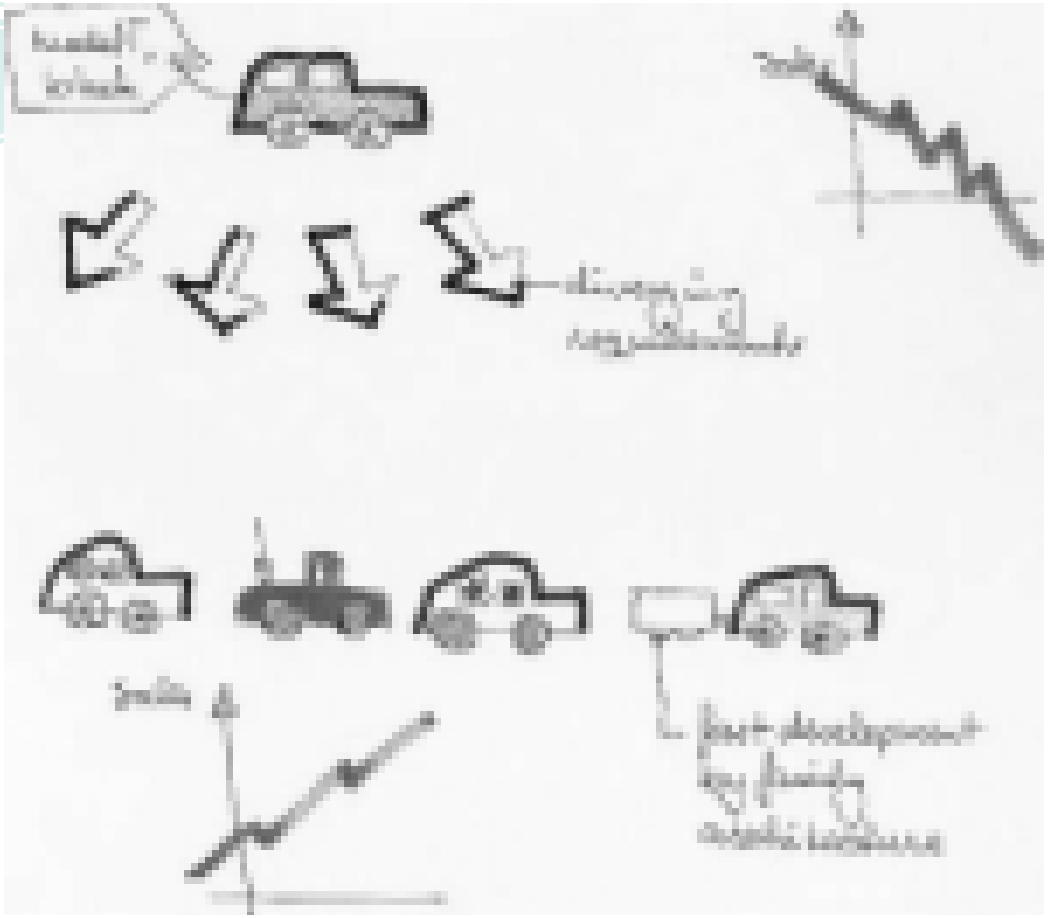
2-Passenger Coupé, Equipped with 3 Oil Lamps, Tubular Horn and Kit of Tools



2-Passenger Torpedo Runabout, Fully Equipped



6-Passenger Town Car, Equipped with 3 Oil Lamps, Tubular Horn and Kit of Tools



One
Product Fits
All

Single Product for all vs. Individual Product

Which of these belong on the same "Production Line" ?

Platforms

- Definition:
  - **A platform is any base of technologies on which other technologies or processes are built.**
- Prerequisite for Mass Customization
  - Mass customisation means the ability to have an individualised product.
  - Mass customisation needs higher technological investments, higher prices for the individualised products and/or to lower profit margins for the company.
- Need to introduce common platforms
  - Plan ahead which shold be used in the different products
  - E.g. Originally, an automobile platform only consisted of floor panels, a suspension system, and rocker panels. Later more parts were added to the platform.
    - The platform provided a structure for major components determining the body size and the size and type of the engine and transmission.
    - The parts comprising the platform were usually the most expensive subsystem in terms of design and manufacturing preparation costs.
    - The use of the platform for different car types typically led to a reduction in the production cost for a particular car type.
- The platform approach enabled a larger variety of products and to reduce costs at the same time.

Combination of Mass Customisation and Common Platform

- Allows us to reuse a common base of technology and, at the same time, to bring out products in close accordance with customers' wishes.
- We call the resulting software development paradigm **Software Product line engineering.**
  - The systematic combination of mass customisation and the use of a common platform for the development of software-intensive systems and software products.

Engineering Customised Products

- Creating the Platform
  - Preparing for mass customization
  - Commonality first, differences later
- Introducing Flexibility
  - Focus on flexibility
  - Prepare for Variability
  - Sharing platforms between product lines
- Reorganising the company
  - Migrating from single system to mass customization need to be supported by proper management
    - Establishing new units, standardize the process/procedurs, workflows, etc

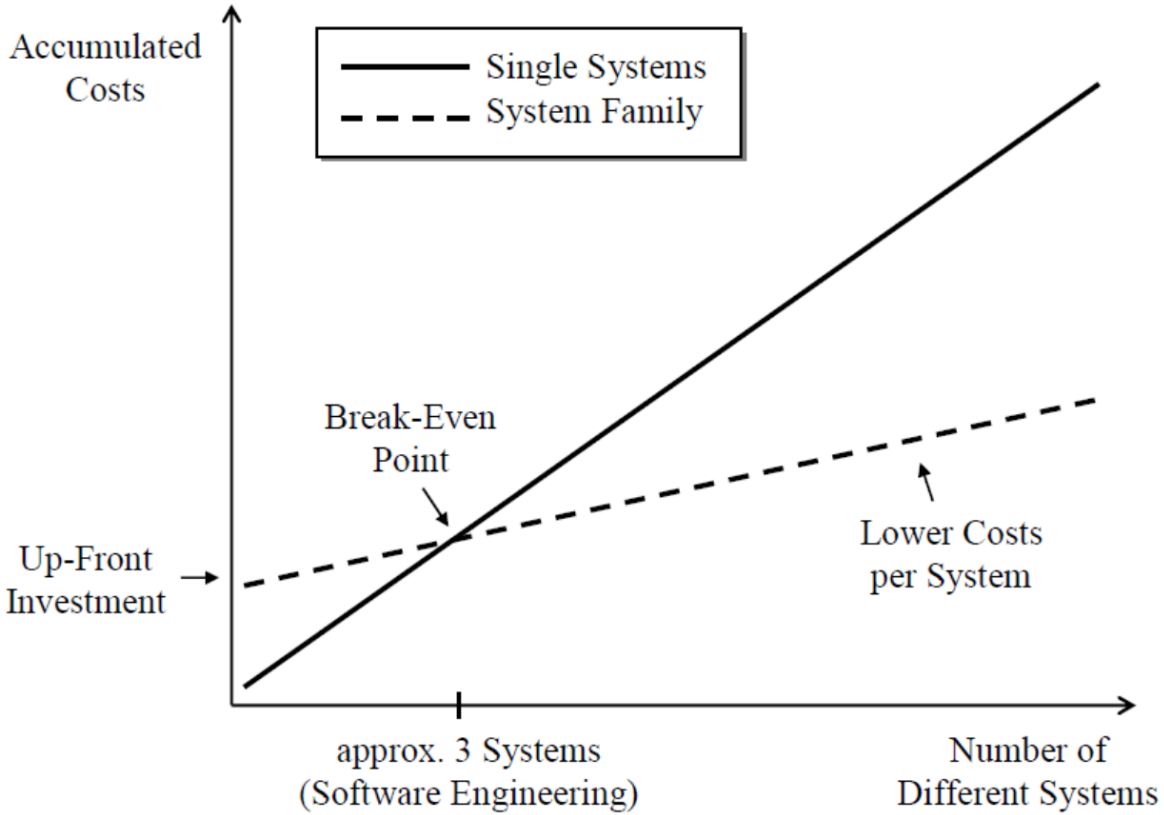
Flexibility

- Artefacts must be sufficiently adaptable to fit into different systems produced in the product line
  - Identify where the product line may differ in the features, the requirements, terms, etc
- Flexibility comes with a set of constraint
  - Some features/requirements may not be fulfilled
- This flexibility is a precondition for mass customization
  - Need to pre-define what possible realisations shall be developed
  - Need to define exactly the places where the products can differ so that they can have as much in common as possible.
- Flexibility is also called "VARIABILITY"

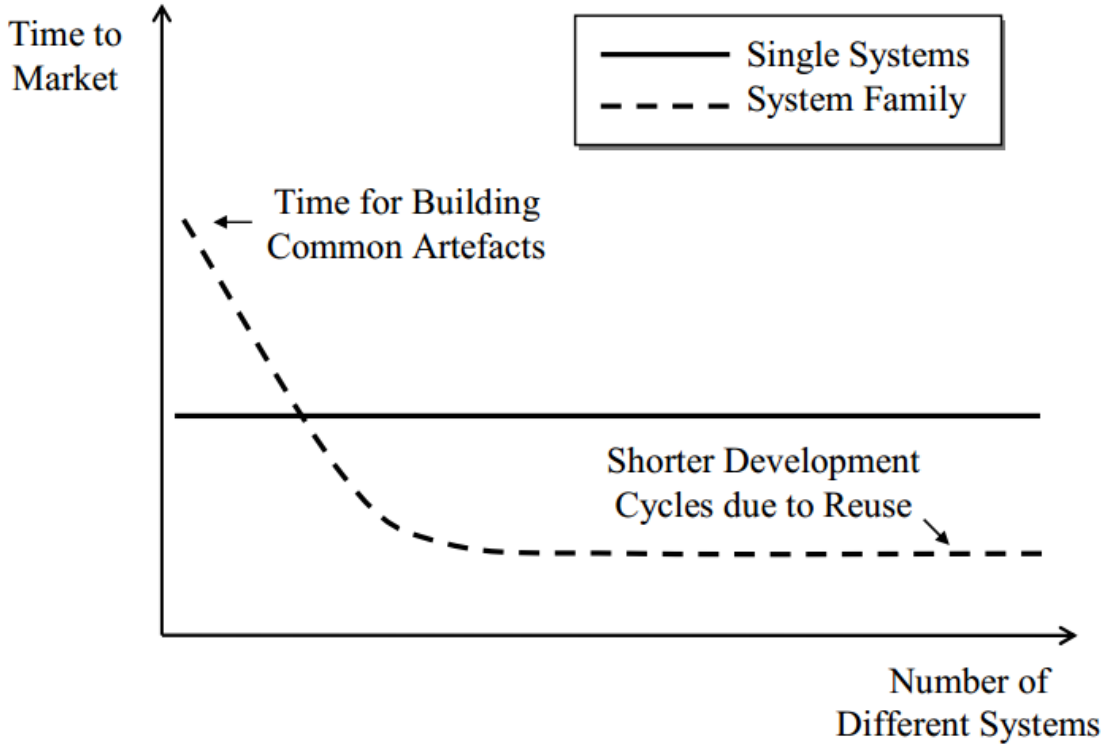
Motivations for Product Line Engineering

- Development Cost Reduction
  - However need up-front investment
  - Usually pay-off around three systems
- Quality Enhancement
- Time to market reduction
- Maintenance Effort Reduction
- Better Development Organization on Software Evolution
- Coping with complexity
- Complexity by Hardware to Software
- Cost Estimation improvement
- Customers benefits
  - Reduced Price
  - Common look and feel

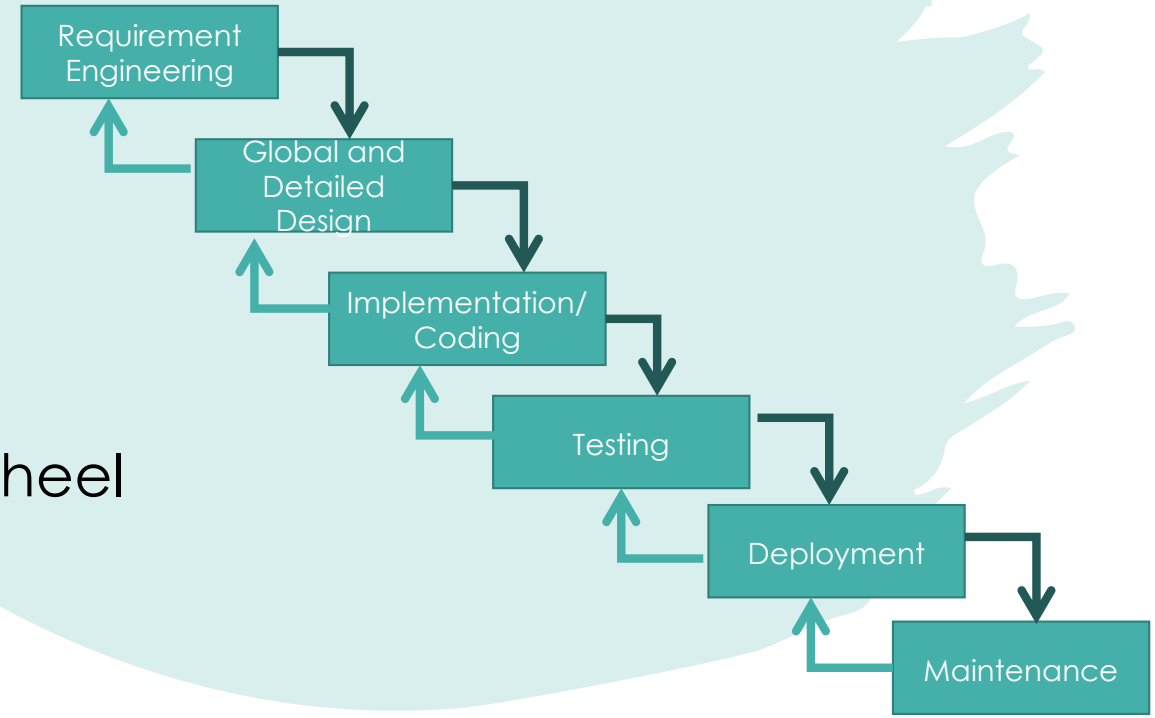
Cost of Development of Single system vs. Product Line Engineering



Time to Market with and without product line engineering



Traditional Software Development Life Cycle



Sequential, chronological, reinventing the wheel over and over

KNOWLEDGE & SOFTWARE ENGINEERING

Software Reuse

Set of systematic, organization-wide measures that are taken to streamline the production and usage of reusable software components in application development

Why REUSE

- Lower cost, better quality, shorter time to market, less process risk

Why NOT Reuse Software

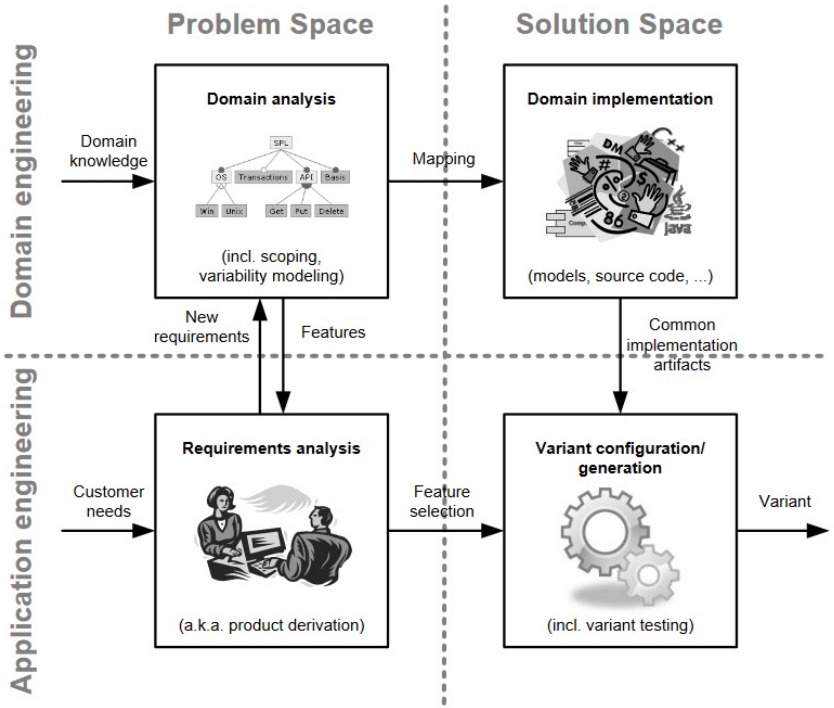
- SW components too information rich.
- Little chance of a match.
- Great variability in user needs.
- Not Invented Here syndrome.
- Architectural mismatches: there is more to a component than function.

•….. *Failure of Reuse as a Broadly Applicable mechanism. Huge reuse libraries went idle…*

Why Not Reuse Software (2)

- SW products have no standard architecture.
  - Analogy with automobiles: standard architectures
  - Cottage industry of parts manufacturers
  - New cars made up, in large parts (98%), of reusable parts

Feature-Oriented Software Development



Sven Apel, Christian K¨astner:An Overview of Feature-OrientedSoftware Development, in Journal of Object Technology, vol. 8, no. 4, July–August 2009,pages 1–36

Product Line Engineering

- A Streamlined Form of Software Reuse
  - Domain specific
  - Centered on an Architecture
  - Captures Domain Knowledge, Assumptions, wisdom, etc

Software product line engineering

REF: POHL, KLAUS, GÜNTER BÖCKLE, AND FRANK J. VAN DER LINDEN. ***SOFTWARE PRODUCT LINE ENGINEERING: FOUNDATIONS, PRINCIPLES AND TECHNIQUES.*** *SPRINGER SCIENCE & BUSINESS MEDIA, 2005.*

KNOWLEDGE & SOFTWARE ENGINEERING

Product Lines

David L. Parnas, 1976, on **Program Families**

- *"**Program families** are defined (analogously to hardware families) as sets of programs whose **common properties** are so extensive that it is **advantageous** to study the common properties to programs before analyzing individual members."*

[Parnas, 1976]

Kyo Kang et al., 1990, on **Feature-Oriented Domain Analysis**

- *"The primary focus of the method is the identification of prominent or distinctive **features** of software systems in a **domain**. These features are **user-visible aspects or characteristics** of the domain. They lead to the creation of a **set of products** that define the domain and also give the method its name: Feature-Oriented Domain Analysis (FODA). The features define both **common aspects** of the domain as well as **differences** between related systems in the domain."*

[Kang et al., 1990]

Product Lines (2)

Paul Clements, Linda Northrop, 2002, on **Software Product Lines**

- *"A **software product line** is a set of software-intensive systems sharing a **common, managed set of features** that satisfy the **specific needs** of a particular market segment or mission and that are developed from a **common set of core assets** in a prescribed way."*

Books

Klaus Pohl, Günter Böckle, Frank van der Linden, 2005, on **Software Product Line Engineering**

- *"**SPLE** is a paradigm to develop software applications (software-intensive systems and software products) using platforms and mass customization."*
- *"A **platform** is any base of technologies on which other technologies or processes are built."*
- *"**Mass customization** is the large scale production of goods tailored to individual customers' needs."*
- *"**Domain engineering** is the process of software product line engineering in which the commonality and the variability of the product line are defined and realized."*

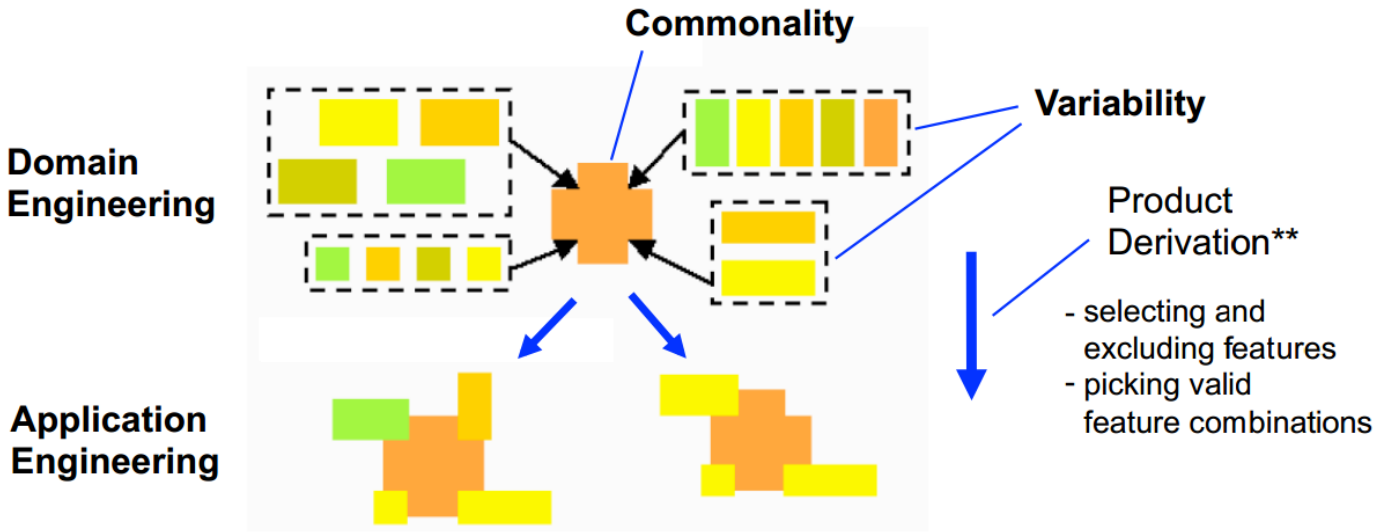
Reinhard Stoiber, 2012 – what my definition would be

- *"A **software product line** (SPL) is a set of software products that all share a significant amount of commonality and differ in their specific configuration of variability."*

Software Product Line Engineering (SPLE)

- Motivations
  - Many systems becoming software-intensive systems
    - Software variability can be implemented more flexibly (than hardware implementation)
- Definition:
  - "Software product line engineering is a paradigm to develop software applications (software-intensive systems and software products) using platforms and mass customization"

Idea



(adapted from Fraunhofer IESE PuLSE™ project (2006))

Framework for SPLE

- Main concepts for building the framework
  - Platforms artefacts
    - Collection of reusable artefacts
    - The artefacts have to be reused in a consistent and systematic way
  - Planning for Reuse
  - Mass customization through variability
    - Must satisfy different stakeholders requirements

Software Platforms

- Definition:
  - A software platform is a set of software subsystems and interfaces that form a **common** structure from which a set of derivative products can be efficiently developed and produced. [Meyer and Lehnerd 1997]
- Software platforms are used to denote the underlying computer system on which application programs can run.
  - This may be the hardware used, e.g. the processor, or, more often, the combination of hardware and operating system
  - E.g. Android as a platform, is used by many product lines of new smart phones

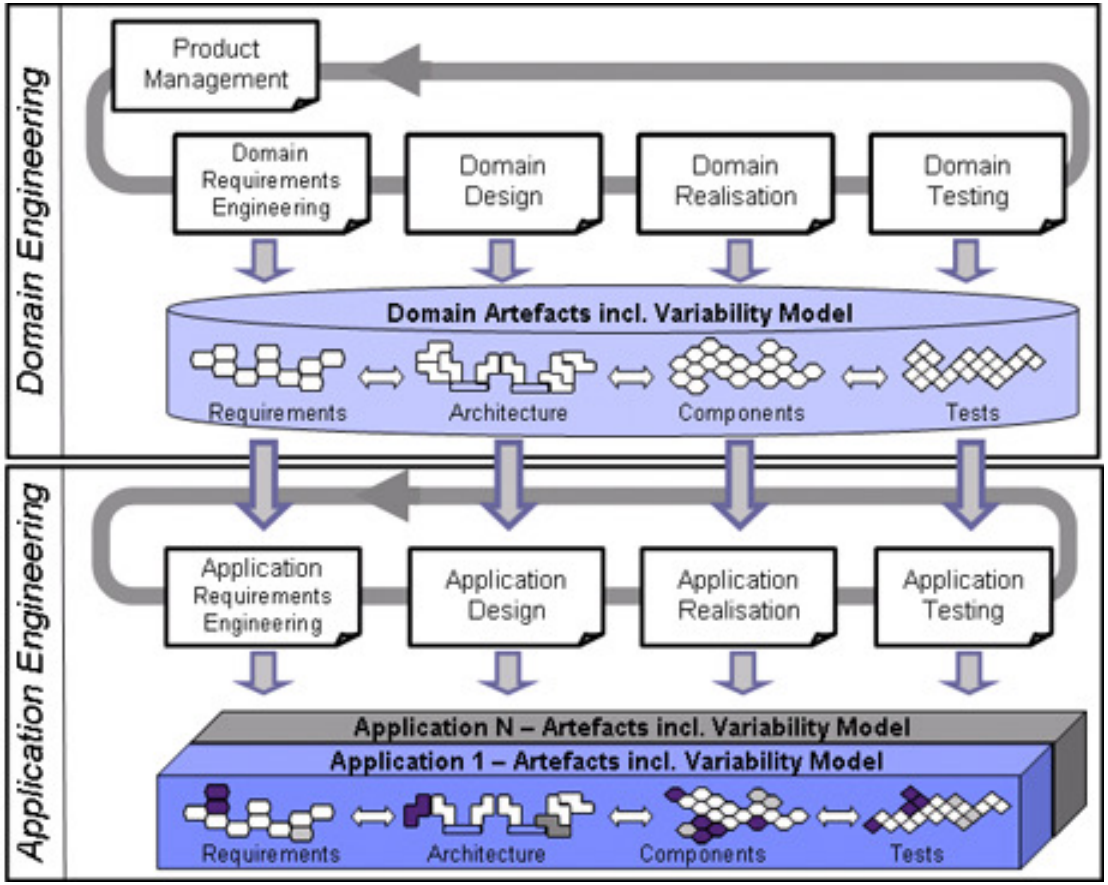
Prerequisites for SPLE

- Proper Enabling Technologies
  - Need adequate technology for applying principles of product line engineering easily
  - OOP: encapsulation, information hiding, etc
  - Component Technology
  - Binding Technique: late binding allows run-time dynamic binding, thus allowing late configuration choices
  - Middleware (Corba, .NET, Java EE)
  - Configuration Management
- Process Maturity
  - Process Models (e.g. CMMI)
  - Requirement Engineering
  - Modeling Techniques (UML, etc)
- Domain Characteristics and Expertise
- Domain Stability

SPLE processes

- Domain Engineering
  - "Domain engineering is the process of software product line engineering in which the commonality and the variability of the product line are defined and realised."
    - Establishment of reusable platforms, and defining commonality and variability of product line
    - The platforms consists of all types of software artefacts with traceable links
      - Requirement, design, realization, test, etc
- Application Engineering
  - "Application engineering is the process of software product line engineering in which the applications of the product line are built by reusing domain artefacts and exploiting the product line variability."
    - Derivation of product line applications from platform established in domain engineering

SPLE Framework



Some Definitions of Artefacts

- **Development Artefact** is
  - Domain artefacts U Application artefacts
  - Development artefacts encompass requirements, architecture, components, and tests.
- **Domain artefacts** are
  - reusable development artefacts created in the sub-processes of domain engineering.
- **Application artefacts** are
  - the development artefacts of specific product line applications.

Domain Artefacts

- Domain artefacts or domain assets compose the platform of the software product line and are stored in a common repository.
- They are produced by the sub-processes of Domain Engineering
- The artefacts are interrelated by traceability links to ensure the consistent definition of the commonality and the variability of the software product line throughout all artefacts.
  - Product Roadmap
  - Domain Variability Model
  - Domain Requirements: Commonality vs Variability
  - Domain Architecture
  - Domain Realisation Artefacts
  - Domain Test Artefacts

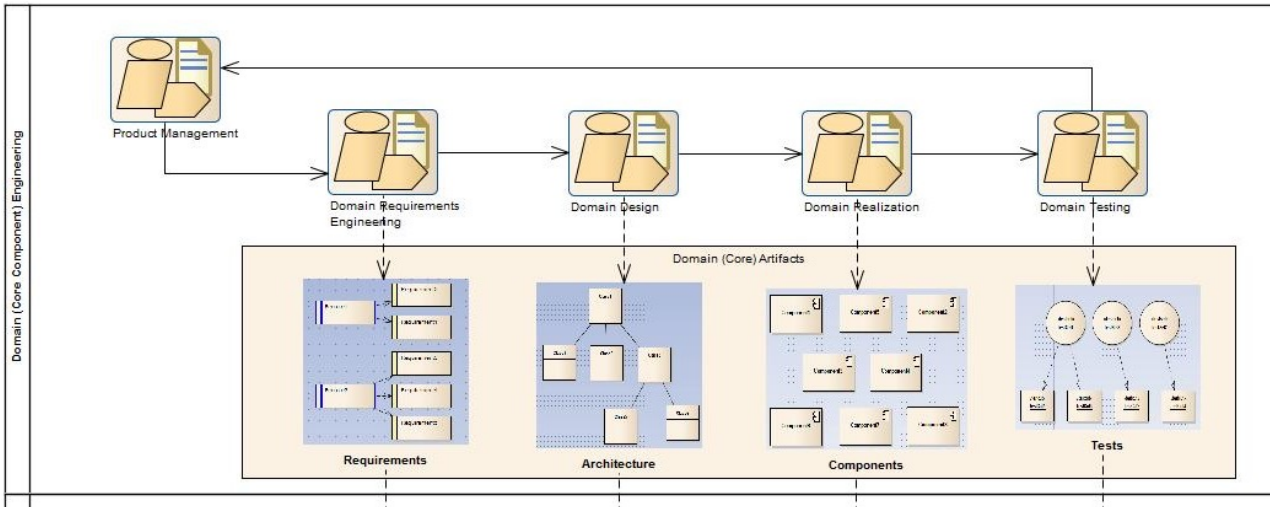
Application Artefacts

- Application artefacts (or application assets) comprise all development artefacts of a specific application including the configured and tested application itself.
  - They are produced by the sub-processes.
- The application artefacts are interrelated by traceability links. The links between different application artefacts are required, for instance, to ensure the correct binding of variability throughout all application artefacts.
- Many application artefacts are specific instances of reusable domain artefacts.
- The orthogonal variability model is used to bind variability in domain artefacts consistently in the entire application.
- Traceability links between application artefacts and the underlying domain artefacts are captured to support the various activities of the application engineering sub-processes.
  - These links also support the consistent evolution of the product line. For example, if a domain artefact changes, the application artefacts affected by this change can be easily determined
- The application artefacts: Application variability model, application requirements, application architecture and application realization artefacts, and application test artefacts

Domain Engineering

Domain Engineering

- Key Goals:
  - Define the commonality and the variability of the software product line.
  - Define the set of applications the software product line is planned for, i.e. define the scope of the software product line.
  - Define and construct reusable artefacts that accomplish the desired variability.



Domain Engineering Sub Processes

- The sub processes are:
  - Product Management
  - Domain requirements engineering,
  - Domain design,
  - Domain realisation, and
  - Domain testing.
- Each sub process has to
  - Detail and refine the variability determined by the preceding subprocess.
  - Provide feedback about the feasibility of realising the required variability to the preceding sub-process.

Product Management

- Related to the economic aspects of the software product line and with the market strategy.
  - The management of the product portfolio of the company or business unit.
  - Product management employs scoping techniques to define what is within the scope of the product line and what is outside.
- INPUT:
  - the company goals defined by top management.
- OUTPUT:
  - a product roadmap that determines the major common and variable features of future products as well as a schedule with their planned release dates.
- Product management provides a list of existing products and/or development artefacts that can be reused for establishing the platform.

Product management for SPL vs Single System

- The platform has an essential strategic meaning for the company.
  - The introduction and elimination of an entire platform have a strong influence on entrepreneurial success.
- A major strength of software product line engineering is
  - the generation of a multitude of product variants at reasonable cost.
- The products in the product portfolio are closely related as they are based on a common platform.
- Product management anticipates prospective changes in features, legal constraints, and standards for the future applications of the software product line and formulates (models) the features accordingly.
  - This means that the evolution of market needs, of technology, and of constraints for future applications is taken into account.

Domain Requirements Engineering

- The domain requirements engineering sub-process encompasses all activities for **eliciting** and **documenting** the common and variable requirements of the product line.
- INPUT:
  - the product roadmap.
- OUTPUT:
  - reusable, textual and model-based requirements and, in particular, the variability model of the product line.
    - the output does not include the requirements specification of a particular application, but the common and variable requirements for all foreseeable applications of the product line.a

Domain RE vs Single systems RE

- The requirements are analysed to identify those that are common to all applications and those that are specific for particular applications
  - i.e. that differ among several applications
- The possible choices with regard to requirements are explicitly documented in the variability model
  - an abstraction of the variability of the domain requirements.
- Based on the input from product management, domain requirements engineering anticipates prospective changes in requirements
  - such as laws, standards, technology changes, and market needs for future applications.

Domain Design

- The domain design sub-process encompasses all activities for defining the reference architecture of the product line.
  - The reference architecture provides a common, high-level structure for all product line applications.
- INPUT:
  - the domain requirements and the variability model from domain requirements engineering.
- OUTPUT:
  - the reference architecture and a refined variability model that includes so-called internal variability
    - e.g. variability that is necessary for technical reasons

Domain Design in SPL vs Single System

- Domain design incorporates **configuration mechanisms** into the reference architecture to support the variability of the product line.
- Domain design considers **flexibility** from the very first, so that the reference architecture can be adapted to the requirements of future applications.
- Domain design defines **common rules** for the development of specific applications based on the reference architecture.
- Domain design designates **reusable parts**, which are developed and tested in domain engineering, as well as application-specific parts, which are developed and tested in application engineering.

Domain Realisation

- The domain realisation sub-process deals with the **detailed design** and the **implementation** of reusable software components.
- INPUT:
  - the reference architecture including a list of reusable software artefacts to be developed in domain realisation.
- OUTPUT:
  - the detailed design and implementation assets of reusable software components.

Domain Realisation in SPL vs Single System

- The result of domain realisation consists of
  - loosely coupled, configurable components, not of a running application.
- Each component is planned, designed, and implemented for the reuse in different contexts, i.e. the applications of the product line.
  - The interface of a reusable component has to support the different contexts.
- Domain realisation incorporates configuration mechanisms into the components (as defined by the reference architecture) to realise the variability of the software product line.

Domain Realisation

- Generate common subsystems
- Create Interfaces that will be used by applications
- Purely abstract, no executables

Domain Testing

- Domain testing is responsible for the validation and verification of reusable components.
  - Domain testing tests the components against their specification, i.e. requirements, architecture, and design artefacts. In addition, domain testing develops reusable test artefacts to reduce the effort for application testing.
- INPUT:
  - domain requirements, the reference architecture, component and interface designs, and the implemented a reusable components.
- OUTPUT:
  - the test results of the tests performed in domain testing as well as reusable test artefacts.

Domain Testing of SPL vs Single system

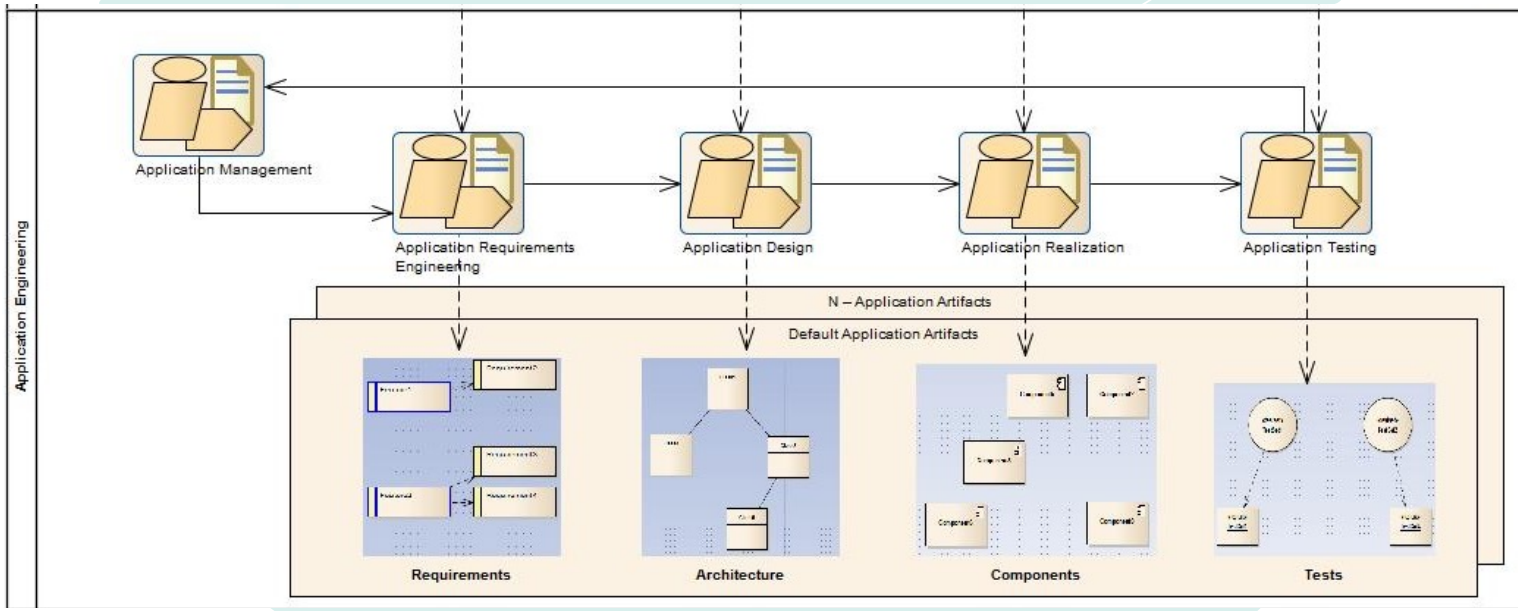
- There is no running application to be tested in domain testing.
  - Indeed, product management defines such applications, but the applications are available only in application testing.
  - Only single components and integrated chunks composed of common parts can be tested in domain testing.
- Domain testing can embark on different strategies with regard to testing integrated chunks that contain variable parts.
  - It is possible to create a sample application, to predefine variable test artefacts and apply them in application testing, or to apply a mixture of the former two strategies.

Application Engineering

Application Engineering

- Key Goals
  - Achieve an as high as possible reuse of the domain assets when defining and developing a product line application.
  - Exploit the commonality and the variability of the software product line during the development of a product line application.
  - Document the application artefacts, i.e. application requirements, architecture, components, and tests, and relate them to the domain artefacts.
  - Bind the variability according to the application needs from requirements over architecture, to components, and test cases.
  - Estimate the impacts of the differences between application and domain requirements on architecture, components, and tests

Application Engineering



Application Requirements Engineering

- The application requirements engineering sub-process encompasses all activities for developing the application requirements specification.
  - The achievable amount of domain artefact reuse depends heavily on the application requirements.
  - Hence, a major concern of application requirements engineering is the detection of deltas between application requirements and the available capabilities of the platform.
- INPUT:
  - the domain requirements and the product roadmap with the major features of the corresponding application.
  - there may be specific requirements (e.g. from a customer) for the particular application that have not been captured during domain requirements engineering.
- OUTPUT:
  - the requirements specification for the particular application

Application RE vs Single System RE

- Requirements elicitation is based on the communication of the available commonality and variability of the software product line.
  - Most of the requirements are not elicited anew, but are derived from the domain requirements.
- During elicitation, deltas between application requirements and domain requirements must be detected, evaluated with regard to the required adaptation effort, and documented suitably.
  - If the required adaptation effort is known early, trade-off decisions concerning the application requirements are possible to reduce the effort and to increase the amount of domain artefact reuse.

Application Design

- The application design sub-process encompasses the activities for producing the application architecture.
  - Application design uses the reference architecture to instantiate the application architecture.
  - It selects and configures the required parts of the reference architecture and incorporates applicationspecific adaptations.
  - The variability bound in application design relates to the overall structure of the considered system (e.g. the specific hardware devices available in the system).
- INPUT:
  - the reference architecture and the application requirements specification.
- OUTPUT:
  - the application architecture for the application at hand.

Application Design in SPL vs Single System

- Application design does not develop the application architecture from scratch, but derives it from the reference architecture by binding variability,
  - i.e. making specific choices at places where the reference architecture offers different variants to choose from.
- Application design has to comply with the rules defined in the texture of the reference architecture.
  - The rules cover the binding of variability as well as the incorporation of application-specific adaptations.
- Application design must evaluate the realisation effort for each required adaptation and may reject structural changes that would require a similar effort as for developing the application from scratch.

Application Realisation

- The application realisation sub-process creates the considered application.
  - The main concerns are the selection and configuration of reusable software components as well as the realisation of application-specific assets.
  - Reusable and application-specific assets are assembled to form the application.
- INPUT:
  - the application architecture and the reusable realisation artefacts from the platform.
- OUTPUT:
  - A running application together with the detailed design artefacts.

Application Realisation in SPL vs Single System

- Many components, interfaces, and other software assets are not created anew.
  - Instead, they are derived from the platform by binding variability.
  - Variability is bound, e.g. by providing specific values for componentinternal configuration parameters.
- Application-specific realisations must fit into the overall structure, e.g. they must conform to the reusable interfaces.
  - Many detailed design options are predetermined by the architectural texture.
  - Applicationspecific components can often be realised as variants of existing components that are already contained in the platform.

Application Testing

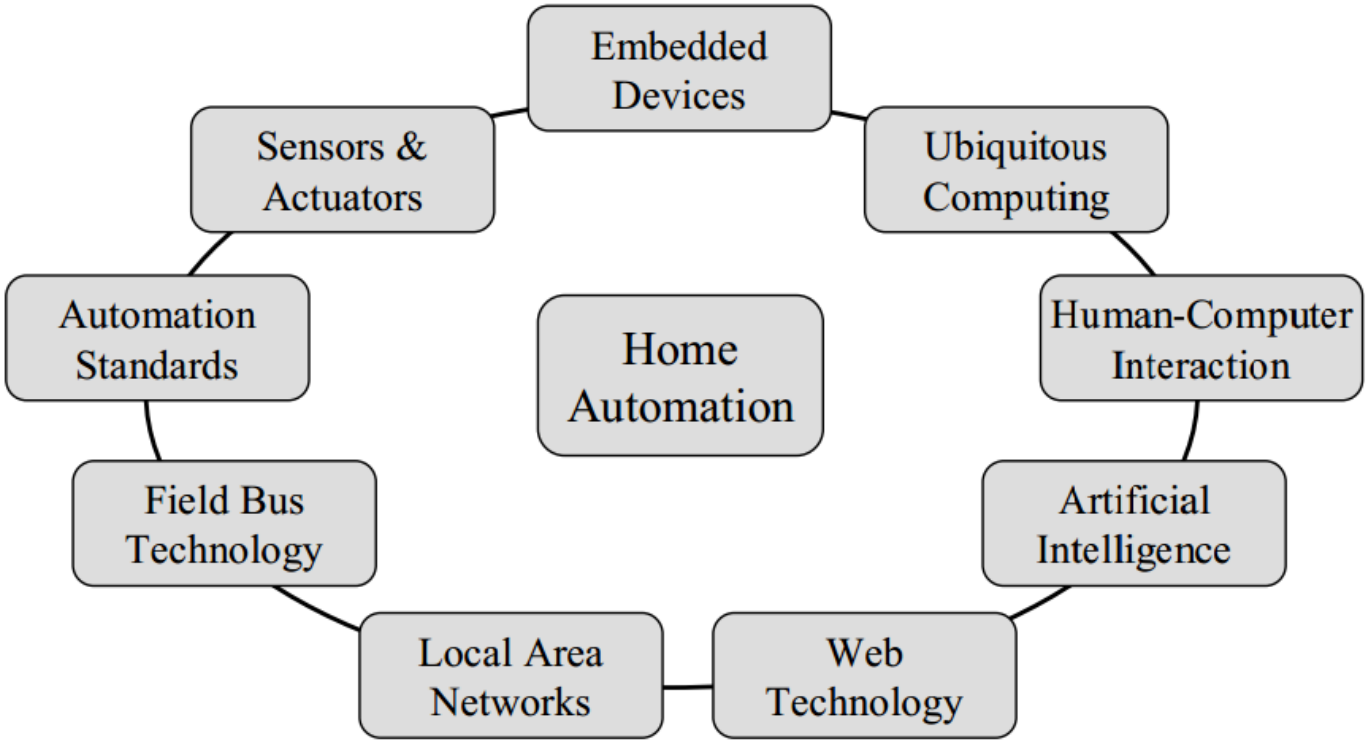
- The application testing sub-process comprises the activities necessary to validate and verify an application against its specification.
- INPUT:
  - all kinds of application artefacts to be used as a test reference, the implemented application, and the reusable test artefacts provided by domain testing.
- OUTPUT:
  - a test report with the results of all tests that have been performed. Additionally, the detected defects are documented in more detail in problem reports.

Application Testing in SPL vs Single System

- Many test artefacts are not created anew, but are derived from the platform.
  - Where necessary, variability is bound by selecting the appropriate variants.
- Application testing performs additional tests in order to detect defective configurations and to ensure that exactly the specified variants have been bound.
- To determine the achieved test coverage, application testing must take into account the reused common and variable parts of the application as well as newly developed application-specific parts.

Example Domain: Home Automation

Domain Related to Home Automation



Goals

- Comfort,
- Security,
- Life safety,
- Low costs

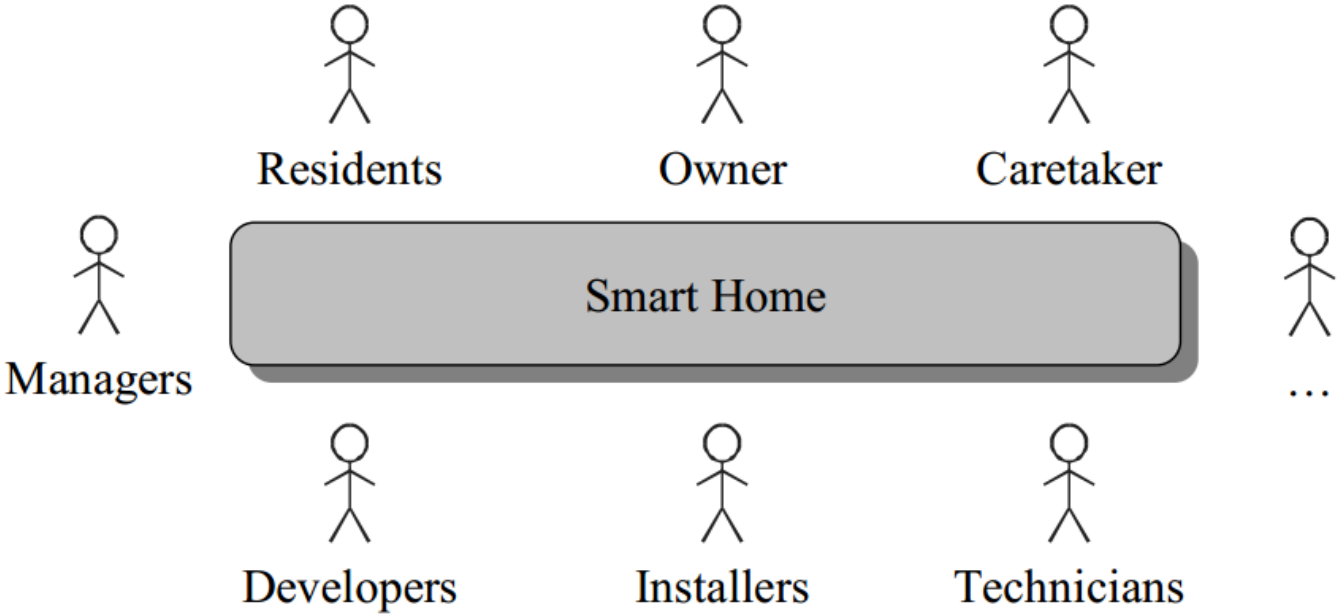
Automating tedious tasks

Surveillance cameras

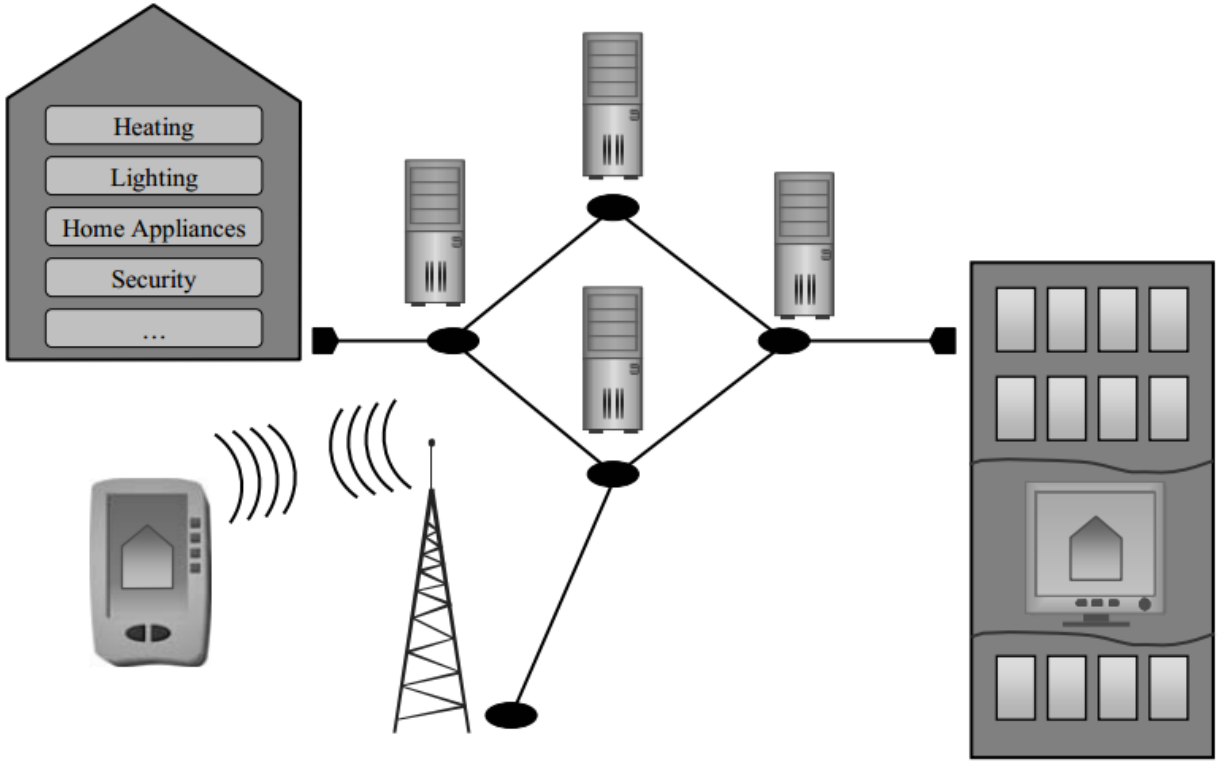
Smoke sensors

Low development and installation cost, low energy consumption

Stakeholders

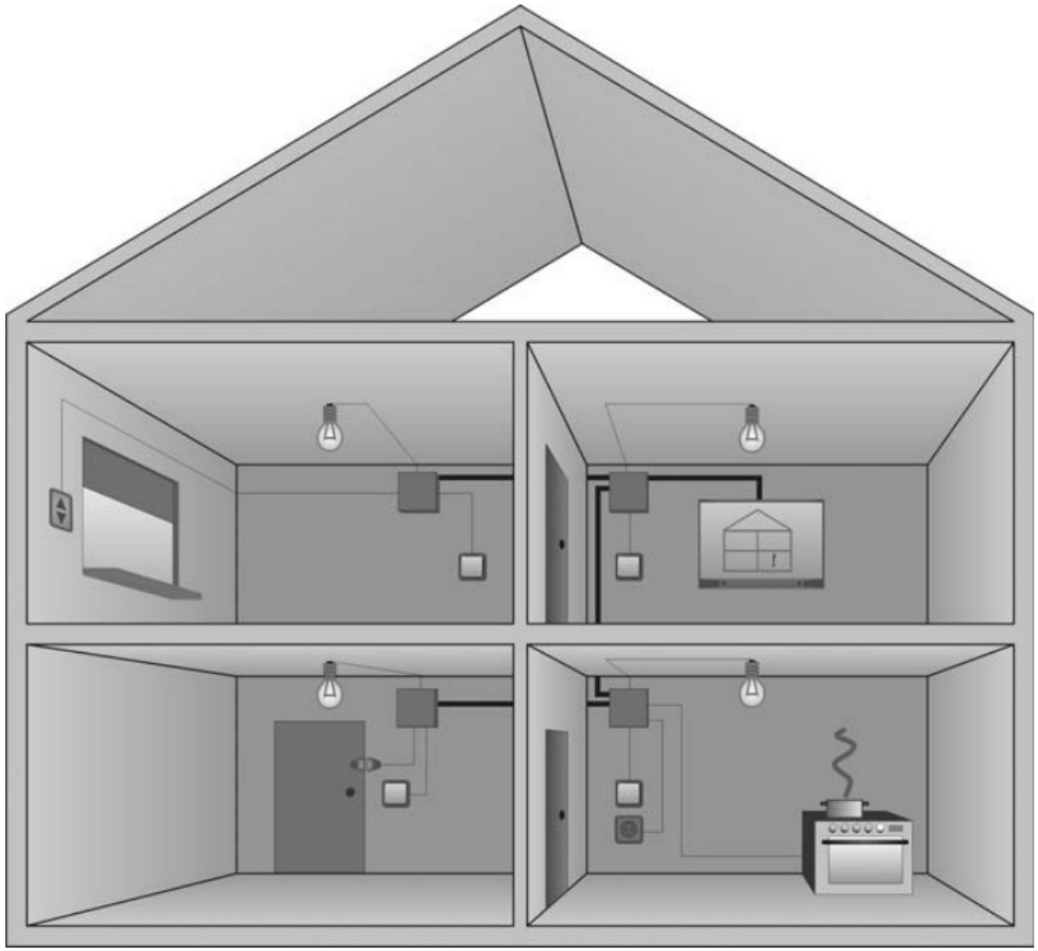


Remote Access to The Smart Home

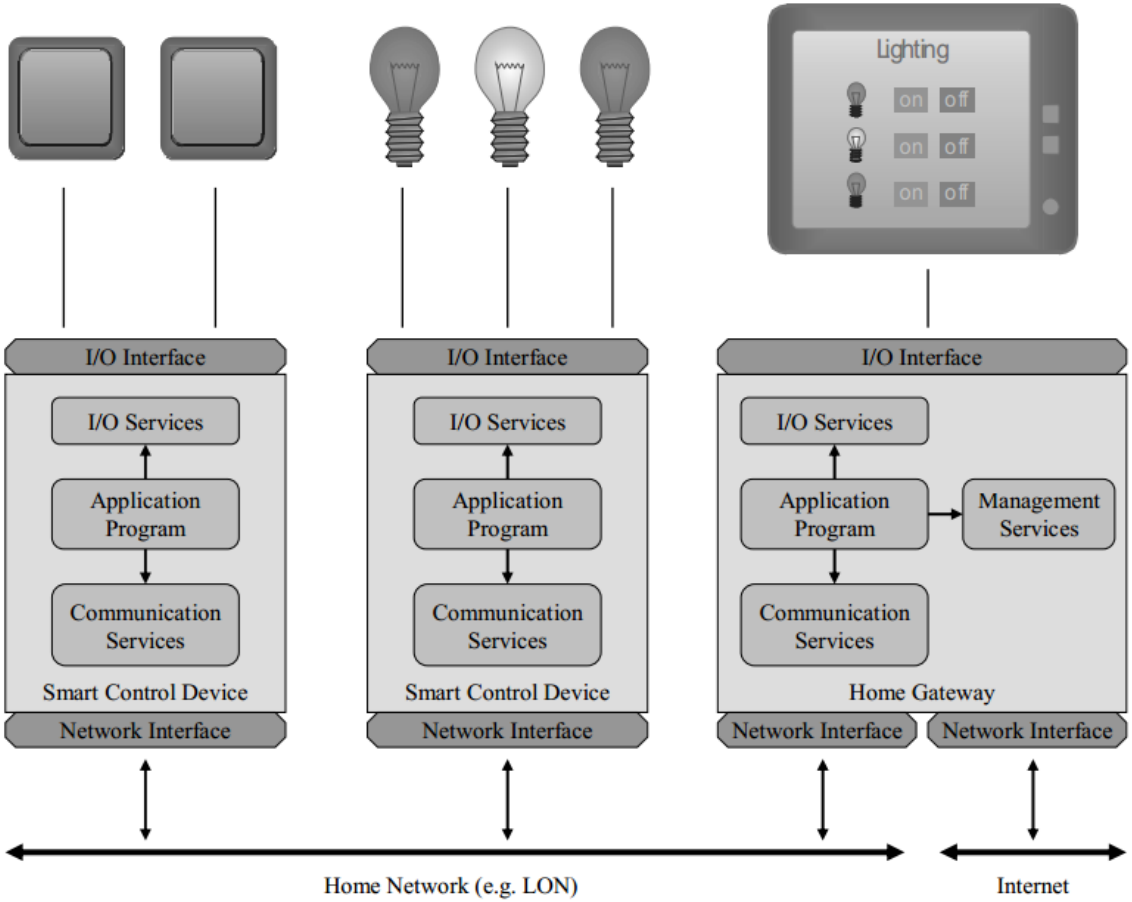


Building Blocks

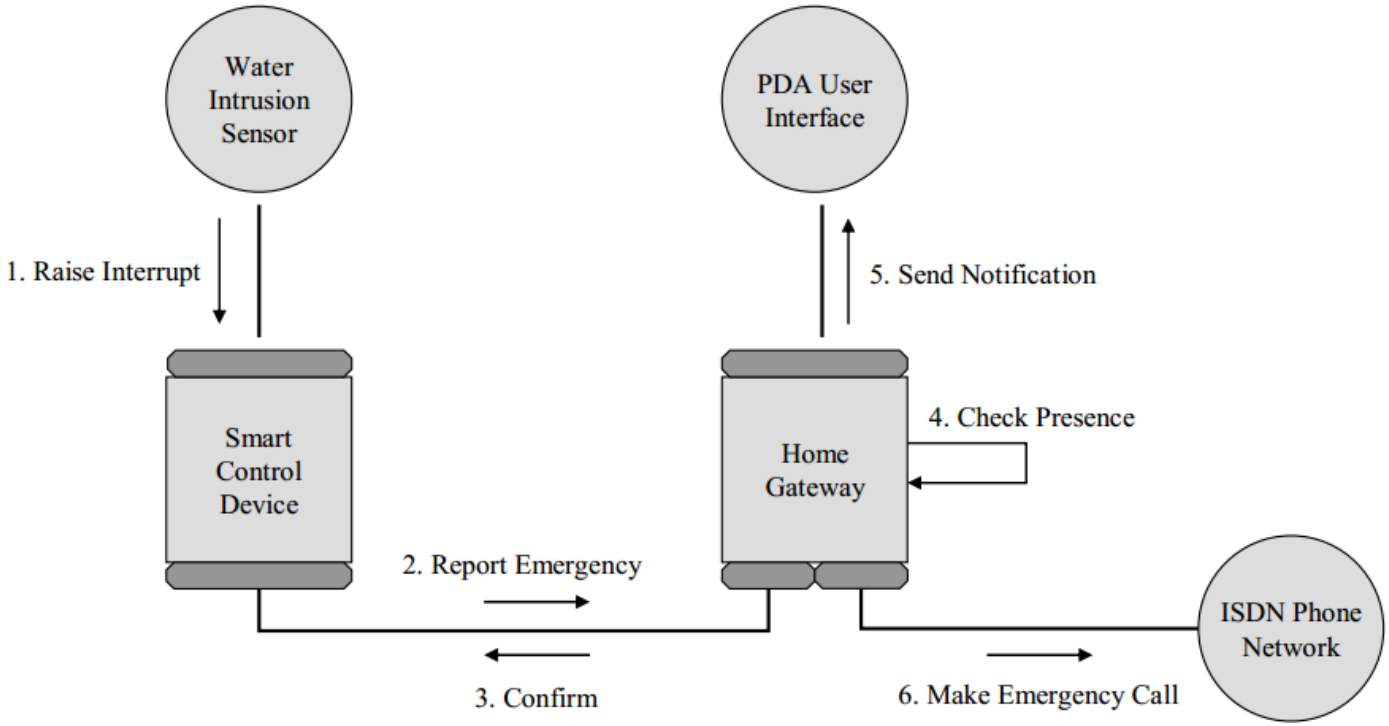
- Sensors and actuators
  - Infrared sensors, light actuators
- Smart Control Devices
  - Read data from sensors, process, and activate the actuators
  - Server is needed to control task
- Home Gateway
  - Central server
  - Database for users and other informations
  - Workflows
  - Other functions: auvi entertainment, Internet, etc
- Networking



Configuration of a home automation system



Scenario with internal system interactions



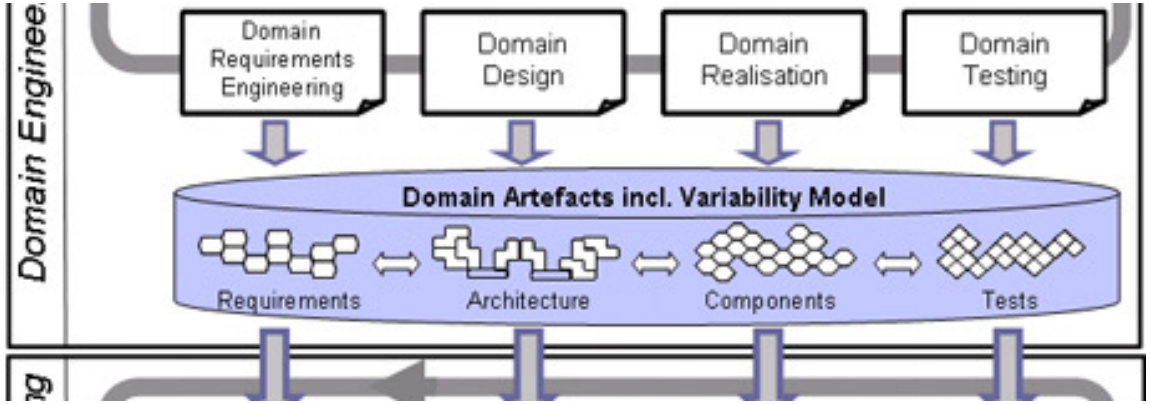
Variability in Smart Home Applications

- User Interface variability
  - Different remote and local UI
- Sensors – different algorithm control
- Different levels of fail-safety
- Different kinds of assistance
  - Assistance for older/handicapped people

Variability

Introduction

- Variability referes to the ability or the tendency to change
- Variability modeling is needed to support the development and the reuse of variable development artefacts
  - The model is defined during Domain Engineering
  - The application engineering exploits this model by binding the appropriate variants



Real World Variability

- An electric bulb can be lit or unlit.
- A software application can support different languages.
- Chewing gum can be sweet or sour.
- A triple band mobile phone supports three different network standards.

Definitions

- A **variability subject** is a variable item of the real world or a variable property of such an item
  - What does vary? – need to identify precisely the variable item/property of the real world
- **A variability object** is a particular instance of a variability subject.
  - How does it vary? –deals with the difference shapes a variability subject can take

Examples

- COLOUR
  - The variability subject "colour" identifies a property of real-world items.
  - variability objects for this variability subject are red, green, and blue.
- PAYMENT METHOD
  - Payment method is a variability subject
  - Variability objects of payment are:
    - payment by credit card, payment by cash card, payment by bill, and payment by cash are examples of variability objects.

Example: HOME SECURITY SYSTEM

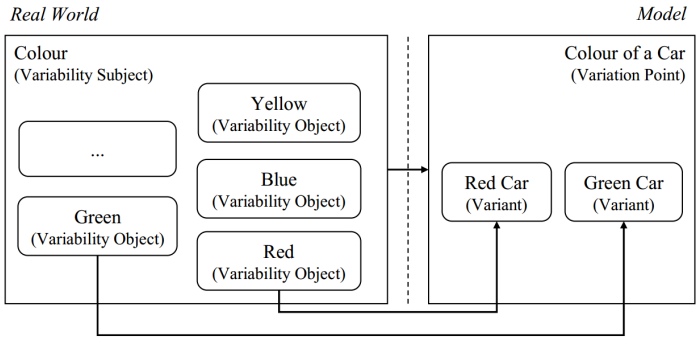
- The identification mechanism of a home security system is a variability subject
- Keypad and Fingerprint scanner are variability objects

Variability in SPLE

- In SPLE, variability subjects and their objects are embedded into the context of a SPL
- A variation point is a representation of a variability subject within domain artefacts enriched by contextual information.
  - The definition applies to all kinds of development artefacts
    - Requirements, architecture, design, code, tests
- Variant is a representation of a variability object within domain artifacts
  - In analogy to the term variation point

Color of a Car as Variation Point

- The variability subject 'color' has several variability objects ('red', 'blue', 'green', 'yellow', etc.).
  - An automotive company wants to build cars in different colors, therefore a variation point "color of a car" (car is the context of the variation point)
  - An automotive company may build only red and green cars
    - therefore only the variants 'red (car)' and 'green (car)' are defined.
    - Other variability objects ('yellow', etc.) are not considered as variants for the automotive company, e.g. for marketing reasons.



Steps to define Variability in SPL

1. Identify variability subject
2. Define variantion point
3. Define variant

Step 1: Identify variability subject

- The engineers of a home automation software product line suggest different ways of communication between system components,
  - e.g. cabled LAN, wireless LAN, Bluetooth, or power line.
- The engineers' suggestions yield the variability subject "type of network".

Step 2: Define variation point

- From the variability subject "type of network". The resulting variation point for the home automation system is "**Home automation system network**".
  - The variation point indicates that the home automation product line has to support different kinds of networks, yet without stating which ones.
  - The development artefacts to be developed are already affected by the existence of the variation point.
  - For example, it may be necessary to introduce an architectural layer that provides a common view of different types of networks.

Step 3: Define Variant

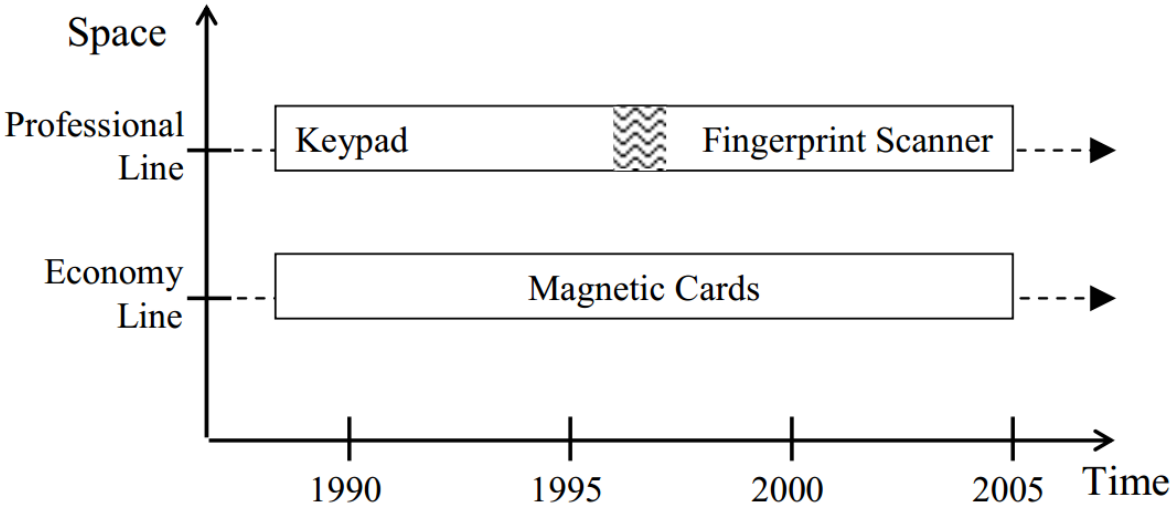
- Possible variability objects for the variability subject "type of network".
- The engineers select cabled LAN and wireless LAN and define them as variants of the variation point "home automation system network".
- The variants make clear that the software to be developed has to support cabled LAN as well as wireless LAN.
  - This allows engineers to develop the software far more efficiently than would be possible knowing only that there are different kinds of networks.
  - Anyway, the engineers still have to prepare for future changes in the set of variants, i.e. the addition of other kinds of networks.

Variability in Time vs. Space

- Variability in time is the existence of different versions of an artefact that are valid at different times.
  - The engineers of a home automation system expect technological progress in identification mechanisms.
  - Therefore they define a variation point "door lock identification mechanism" with just one variant "magnetic card".
  - Later, when sufficiently reliable fingerprint scanners appear on the market, the engineers replace the variant "magnetic card" by the variant "fingerprint scanner".

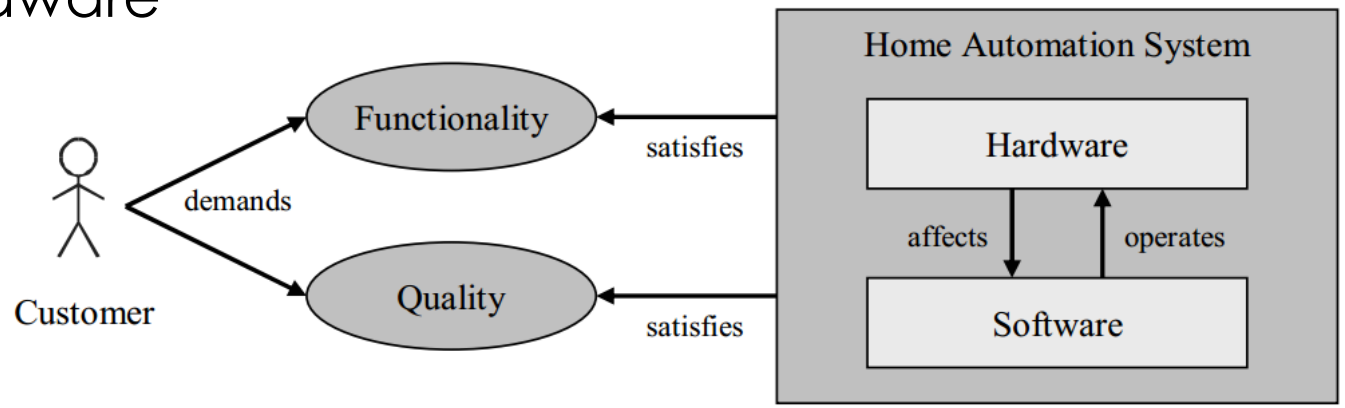
- Variability in space is the existence of an artefact in different shapes at the same time.
  - A home automation system offers the variation point "system access by" with four variants: web browser, mobile phone (SMS), telephone call (computer voice), and secure shell client (SSH).
  - These variants are associated, for instance, with requirements artefacts of the software product line.

Example of a Home Automation System

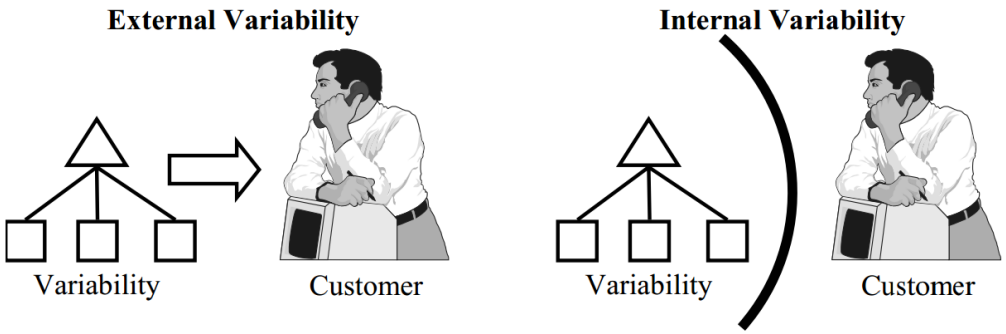


Causes of Variability

- Customer wishes and technical constraints
- Functionality and quality
- Different customer needs
- Hardware



Internal vs. External Variability



- External variability is the variability of domain artefacts that is visible to customers.
  - Caused by: stakeholder needs, laws/standards
- Internal variability is the variability of domain artefacts that is hidden from customers.
  - Caused by: technical refinements, technical reasons

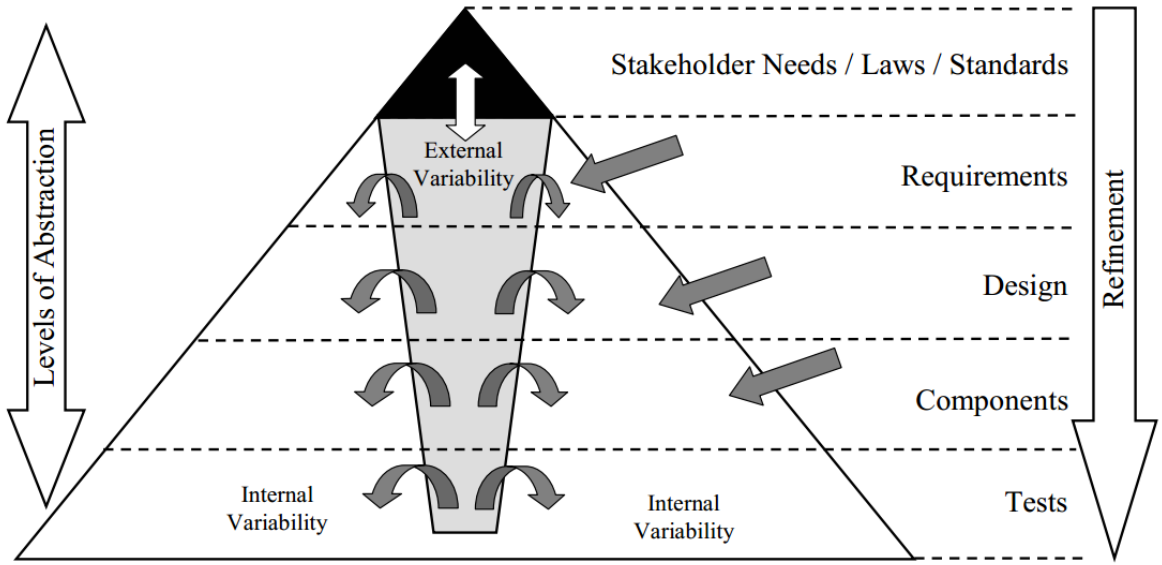
Example: Home Automation System

- External Variability
  - The customers of a home automation system can choose between three electronic door lock identification mechanisms: keypad, magnetic card, and fingerprint scanners.
- Internal Variability
  - The communication protocol of a home automation system network offers two different modes: one is optimised for high bandwidth, the other for error correction.
  - The installers measure the quality of the available communication medium and choose the mode based on the measurement results.

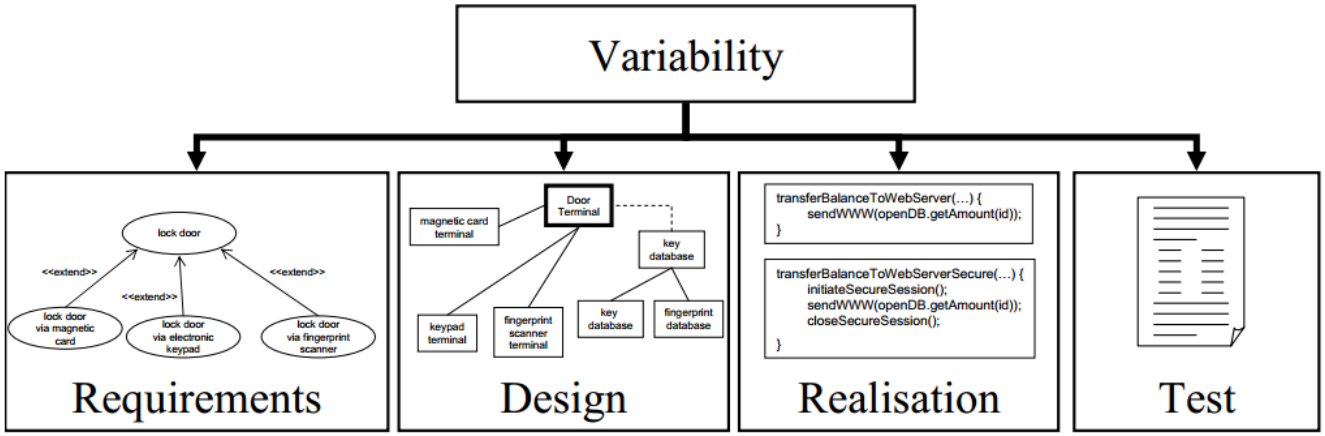
Variability Pyramids

- Increased Complexity
- Increased Interrelations

Variability pyramid

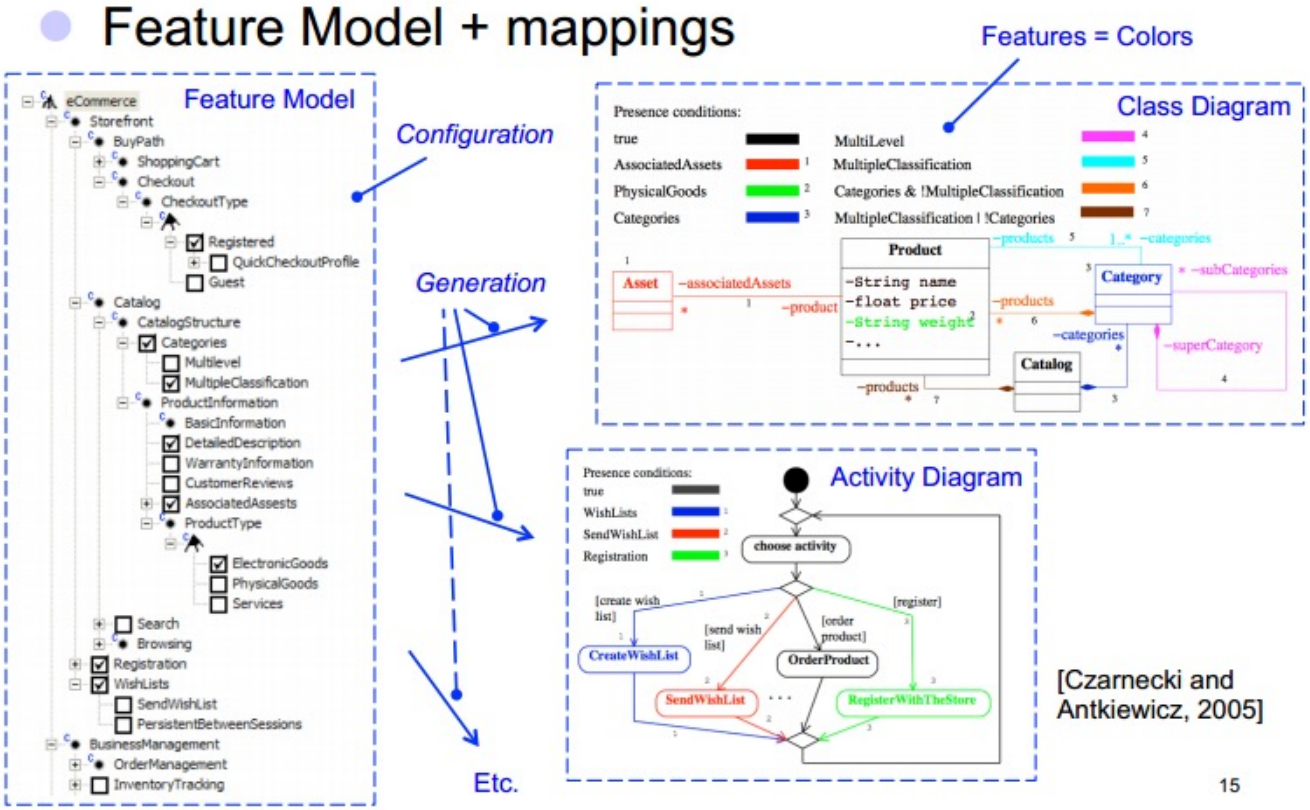


Variability must be traceable in all artefacts



Variability Modeling Using Feature Model + Mappings

- Feature Model + mappings



[Czarnecki and Antkiewicz, 2005]